Internship Progress Dashboard Deployment Document

📘 Introduction

This document provides a comprehensive, step-by-step guide for deploying the **Internship Progress Dashboard** application. It covers everything from local environment setup using Docker to production-ready deployment strategies on cloud platforms.

Purpose of Deployment

- To establish a production environment for the application, ensuring high availability, security, and performance.
- To standardize the deployment process for consistency across development, staging, and production environments.
- To provide an easy-to-follow guide for any DevOps or system administrator.

Overview of System

The Internship Progress Dashboard is a full-stack web application designed for managing intern tasks and tracking performance.

Component	Technology	Role
Frontend	React.js (Vite) + Tailwind CSS	User interface for interns and mentors
Backend	FastAPI (Python)	RESTful API for business logic and data access
Database	MongoDB (NoSQL)	Persistent storage for user and task data
Security	JWT	Stateless user authentication and authorization
Containerization	Docker	Packaging the application for reliable deployment

🧱 System Architecture

The application follows a three-tier, containerized architecture.

- **Frontend (React App):** Served via a static file server or a dedicated web server container. It communicates with the Backend API using HTTP requests.
- **Backend (FastAPI Server):** A Python-based server that handles all business logic, manages authentication, and interfaces with the database.
- **Database (MongoDB):** The persistent data store. In a production setting, this is typically hosted by a managed service (e.g., MongoDB Atlas).

Connection Flow (Client → API → DB)

1. **Client (Browser):** The user interacts with the React frontend.
2. **Frontend (React):** Makes an API request to the Backend server (e.g., `POST /tasks`).
3. **Backend (FastAPI):**
    - Validates the incoming request (including JWT authentication).
    - Processes the request using application logic.
    - Interacts with the MongoDB Database (e.g., `db.collection.insertOne(...)`).
    - Returns a response to the Frontend.
4. **Frontend (React):** Renders the response data for the user.

⚙️ Environment Setup

Required Tools

Install the following tools before starting the deployment process:

- **Node.js (LTS Version):** Required to build the React frontend.
    - *Verification Command:* `node -v`
- **Python (3.8+):** Required for the FastAPI backend.
    - *Verification Command:* `python3 --version`
- **Docker & Docker Compose:** Essential for containerizing the entire application stack.
    - *Verification Command:* `docker --version` and `docker compose version`
- **Git:** For cloning the source code repository.
    - *Verification Command:* `git --version`

Environment Variables

The application requires specific configuration variables to connect components and ensure security. These should be set in a `.env` file (for local development) or managed securely by the cloud provider (for production).

Variable Name	Purpose	Example Value
`MONGO_URI`	Connection string for the MongoDB instance.	`mongodb://db:27017/dashboard` (Local)
`JWT_SECRET`	A long, complex, and unique secret key for signing JWTs.	`aVeryStrongAndRandomSecretKey123!`
`API_BASE_URL`	The public URL of the backend API, used by the frontend.	`https://api.dashboard.com` (Production)

🐳 Docker Setup

This project uses Docker to create repeatable and isolated environments for each service.

Dockerfile (Frontend)

This file builds the production static assets for the React application.# Stage 1: Build the React application

FROM node:18-alpine as build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

Stage 2: Serve the static build files

FROM nginx:stable-alpine

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

Dockerfile (Backend)

This file sets up the Python environment and runs the FastAPI server using a production-ready WSGI server (e.g., Uvicorn/Gunicorn).FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

Gunicorn is used with uvicorn workers for production

CMD ["gunicorn", "app.main:app", "--workers", "4", "--worker-class", "uvicorn.workers.UvicornWorker", "--bind", "0.0.0.0:8000"]

`docker-compose.yml`

This file orchestrates the multi-container application locally.version: '3.8'

services:

frontend:

build: ./frontend

container_name: dashboard-frontend

ports:

- "80:80"

depends_on:

- backend

environment:

- API_BASE_URL=http://localhost:8000 # Use backend service name and port

```
  backend:
    build: ./backend
    container_name: dashboard-backend
    ports:
      - "8000:8000"
    depends_on:
      - database
    environment:
      - MONGO_URI=mongodb://database:27017/intern_db
      - JWT_SECRET=${JWT_SECRET} # Read from host environment or .env file
  database:
    image: mongo:5.0
    container_name: dashboard-mongodb
    volumes:
      - mongo-data:/data/db
    ports:
      - "27017:27017" # Exposed for local debugging only
volumes:
  mongo-data:
```

Explanation of Docker Services

Service Name	Description	Ports Mapping	Volume
`frontend`	Builds the React app and serves it via NGINX.	`80:80` (Host:Container)	None
`backend`	Runs the FastAPI application with Gunicorn/Uvicorn.	`8000:8000` (Host:Container)	None
`database`	Uses the official MongoDB image.	`27017:27017` (Host:Container)	`mongo-data` for persistent storage

🛠️ Local Deployment Steps

Follow these steps for a quick, all-in-one local deployment using Docker Compose.

Step 1: Clone Repository

Fetch the source code from the version control system.git clone [repository-url]

cd Internship-Progress-Dashboard

Step 2: Setup Environment Variables

Create a file named `.env` in the root of the project directory and populate it with the required environment variables.# .env file content example

JWT_SECRET=aVeryStrongAndRandomSecretKey123!

Note: MONGO_URI is set internally in docker-compose.yml for local use

Step 3: Run Docker Compose

Execute the build and run commands for the containers. The `-d` flag runs them in detached mode.docker compose up --build -d

Step 4: Access Application

Wait for a few moments for all services to start (check logs with `docker compose logs`).

- **Frontend Access:** Open your browser to `http://localhost:80`
- **Backend API Check:** Test the backend health endpoint at `http://localhost:8000/docs`

☁️ Cloud Deployment

For production deployment, two primary options are recommended, depending on complexity and control requirements.

Option 1: Render / Railway (Managed Platform)

This option is beginner-friendly, fast, and scalable for quick production launches.

1. **Deploy Backend (FastAPI):**
    - Connect your Git repository to Render/Railway.
    - Configure the service to use the Backend `Dockerfile` and expose port `8000`.
    - Set the `MONGO_URI` and `JWT_SECRET` securely in the platform's environment variables.
2. **Connect MongoDB Atlas:** Update the `MONGO_URI` in the backend service to point to the secure connection string provided by MongoDB Atlas (see Database Deployment section).
3. **Deploy Frontend (React):**
    - Configure a second service (Static Site or Web Service) to use the Frontend `Dockerfile`.
    - Set the `API_BASE_URL` in the frontend environment variables to the public URL of the deployed backend service.

Option 2: AWS (Advanced)

This option offers maximum control and optimization for enterprise-level scaling.

Component	AWS Service	Purpose
Backend	EC2 + ECS/Fargate	Hosting the FastAPI container in a scalable, secure manner.
Frontend	S3 + CloudFront	Hosting static files (React build) globally with low latency.
Database	MongoDB Atlas	Managed, highly available database service.

Steps:

1. **Backend (EC2/ECS):**
    - Provision an EC2 instance or set up an ECS Cluster/Fargate Task.
    - Build and push the Backend Docker image to Amazon ECR.
    - Deploy the container, ensuring secure access to MongoDB Atlas.

    - Place an Application Load Balancer (ALB) in front for traffic distribution.
2. **Frontend (S3 + CloudFront):**
    - Run `npm run build` locally to generate static files.
    - Upload the contents of the `dist` folder to an S3 bucket.
    - Configure an AWS CloudFront distribution to cache and serve the S3 content globally.
3. **DNS & Routing:** Point your domain records (A or CNAME) to the CloudFront distribution (for frontend) and the ALB (for backend).

🗄️ Database Deployment

For production, the database must be managed, secure, and highly available. MongoDB Atlas is the recommended solution.

MongoDB Atlas Setup

1. **Create an Atlas Account:** Sign up for MongoDB Atlas.
2. **Cluster Creation:**
    - Choose a cloud provider (AWS, GCP, or Azure) and a region close to your backend deployment.
    - Select the M10+ tier for production (or M0/M2 for testing/development).
3. **IP Access List:** Add the IP addresses of your backend server(s) or select "Access from Anywhere" (less secure, use only if necessary).
4. **Database User:** Create a dedicated database user with a strong password for the backend application to use.

Connection String

After cluster creation, navigate to the "Connect" section to retrieve the secure connection string (`MONGO_URI`).

- **Format:** `mongodb+srv://<dbuser>:<password>@<cluster-name>.mongodb.net/intern_db?retryWrites=true&w=majority`
- Replace `<dbuser>`, `<password>`, and `<cluster-name>` with your specific Atlas credentials.

🔐 Security Configuration

Production deployment requires rigorous security measures.

HTTPS (SSL)

All traffic between the client and the backend/frontend must be encrypted.

- **Cloud Deployment:** Use a Load Balancer (e.g., AWS ALB) or service provider (Render, CloudFront) that provides free and automatic SSL/TLS certificates (e.g., ACM in AWS).
- **Self-Managed:** If running on a dedicated server, configure a reverse proxy like NGINX or Caddy to handle SSL termination using **Let's Encrypt**.

Secure Environment Variables

- **NEVER** commit secret keys (`JWT_SECRET`, `MONGO_URI` passwords) to Git.
- Use cloud provider secrets management tools (e.g., AWS Secrets Manager, Render/Railway environment variables) to inject secrets securely into the running containers.

JWT Protection

- Ensure the `JWT_SECRET` is at least 32 characters long and complex.
- Implement refresh tokens and enforce short expiration times for access tokens.
- The backend must validate the signature and expiration of every incoming JWT.

📈 CI/CD Pipeline

A Continuous Integration/Continuous Deployment (CI/CD) pipeline automates the build, test, and deployment process, minimizing human error.

GitHub Actions / CI Pipeline

Set up a `main.yml` workflow in GitHub Actions (or equivalent in GitLab CI, Jenkins, etc.).

Stage	Action	Service
CI (Build & Test)	Triggered on every pull request to `main`.	Frontend & Backend
1. Build & Lint	Run `npm install` + `npm run lint/test` (frontend) and `pip install` + `pytest` (backend).	All
CD (Deployment)	Triggered on merge to the `main` branch.	All
2. Docker Build	Build production Docker images for Frontend and Backend.	All

Stage	Action	Service
3. Push to Registry	Push the new, tagged images to a container registry (e.g., Docker Hub, AWS ECR).	All
4. Deploy	Update the production service (e.g., signal Render to pull the new image or update ECS/EC2 instances).	All

Example Deployment Command (After Image Push)

A common action is to tell the cloud service to redeploy:# Example AWS ECS update command in a deployment script

aws ecs update-service --cluster dashboard-cluster --service dashboard-api-service --force-new-deployment

⚠️ Error Handling & Monitoring

Effective monitoring is crucial for a production-ready application.

Logs (Backend Logs)

- Configure FastAPI to output structured logs (JSON format is preferred).
- Ensure logs capture request details, errors, and authentication failures.
- Use a centralized logging system (e.g., ELK stack, CloudWatch Logs, Datadog) to aggregate and search logs from all containers.

Error Tracking

Integrate an error tracking service (e.g., Sentry or Bugsnag) into both the Frontend and Backend.

- **Purpose:** Catches unhandled exceptions, notifies the team, and provides detailed stack traces for rapid debugging.

Basic Monitoring Tools

Tool	Focus	Metrics
Load Balancer	Health Checks	Request Count, Latency, Error Rates (5xx)

Tool	Focus	Metrics
Container Host	Resource Usage	CPU, Memory, Disk I/O
Database (Atlas)	Performance	Query Latency, Connection Pool Usage, Open Connections

🔄 Scaling Strategy

The application must be able to handle increasing load without degradation in performance.

Horizontal Scaling

- **Backend:** This is the easiest layer to scale. Configure the Load Balancer/Service to run multiple identical instances (containers) of the FastAPI backend.
- **Frontend:** The React static files are inherently horizontally scalable via CDN (CloudFront).

Load Balancing

- A Load Balancer (ALB, NGINX Reverse Proxy, or platform-provided) distributes incoming traffic across multiple healthy backend instances.
- Ensure the Load Balancer uses sticky sessions only if strictly necessary, but preferably, keep the application stateless.

Database Scaling

- MongoDB Atlas provides built-in scaling (sharding, replicas).
- For high read traffic, utilize read replicas (secondary members) in the MongoDB cluster.

🧪 Testing Before Deployment

A pre-deployment testing gate in the CI/CD pipeline ensures only stable code reaches production.

API Testing (Integration and Unit Tests)

Use a framework like **Pytest** (Python) to verify that:

- All unit functions work as expected.
- Endpoints correctly handle inputs, authentication, and error conditions.
- Database interactions (CRUD operations) are successful.

UI Testing (End-to-End)

Use a tool like **Cypress** or **Playwright** to simulate user journeys:

- User login/logout flow.
- Task assignment and progress tracking.
- Form submissions and data validation.

Integration Testing (Services Communication)

- Ensure the Frontend can successfully connect to the Backend API.
- Verify that the Backend can successfully connect and transact with the MongoDB Database.

🔮 Future Improvements

These are advanced features for achieving a highly resilient and optimized infrastructure.

- **Kubernetes (K8s) Adoption:**
  - Migrate Docker Compose setup to Kubernetes for advanced container orchestration, self-healing, and declarative deployment management.
- **Auto-Scaling:**
  - Implement **Horizontal Pod Autoscaler (HPA)** in Kubernetes or configure AWS Auto Scaling groups to automatically adjust the number of backend instances based on CPU utilization or request queue length.
- **Advanced Monitoring:**
  - Implement **Prometheus** for metrics collection and **Grafana** for rich, customizable visualization dashboards of application health and performance.